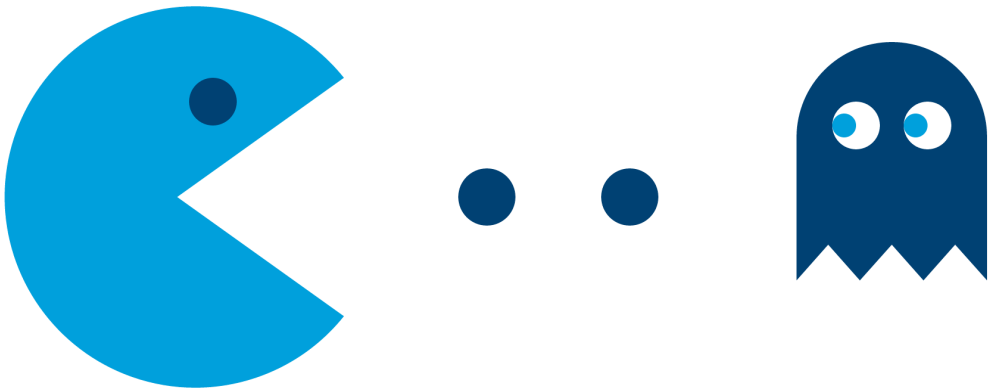
UADER FCyT

TRABAJO FINAL INTEGRADOR
SISTEMA PACMAN POO

Metodología de la Programación
Ingeniería en Telecomunicaciones

Mg. Lourdes Pralong, Esp. Julián Escalante



2026

OBJETIVO GENERAL

Desarrollar una aplicación en Python empleando los conceptos y prácticas trabajados a lo largo del curso:

- Funciones y modularidad.
- Tipos abstractos de datos.
- Programación orientada a objetos (POO).
- Herencia, polimorfismo, abstracción y encapsulamiento.
- Modelado mediante UML.
- Manejo de Interfaz y Persistencia de Datos.

Además, deberán presentar y defender oralmente su proyecto, explicando la estructura, decisiones de diseño y funcionalidades implementadas.

DESCRIPCIÓN DEL PROYECTO

Cada equipo (2 personas) deberá construir un Sistema de Juego Pacman POO, capaz de:

- Controlar la lógica de juego de Pacman e integrar el movimiento autónomo de entidades enemigas (fantasmas).
- Gestionar el estado del juego, colisiones, puntos, vidas y ciclo de partida.
- Aplicar principios de programación orientada a objetos.
- Presentar un diagrama UML que representa las clases y sus relaciones.

REQUERIMIENTOS FUNCIONALES

El sistema debe contar con una interfaz gráfica (se sugiere usar Tkinter[1]) y cumplir con las siguientes funcionalidades:

1. Gestión de Jugadores:
    a. Solicitar e ingresar el nombre del jugador al iniciar la partida.
    b. Asignar un registro único en el historial de puntuaciones.
2. Control y Movimiento de Entidades:
    a. Permitir el control direccional de Pacman mediante las teclas de flecha (Up, Down, Left, Right).
    b. Mover de manera autónoma a los fantasmas dentro de la cuadrícula del mapa.
3. Interacción con el Tablero:

[1] https://docs.python.org/3/library/tkinter.html

a. Detectar la recolección de puntos en la cuadrícula e incrementar la puntuación del jugador.
b. Evitar que el personaje atraviese las paredes del laberinto.

4. Gestión de Colisiones y Vidas:
   a. Detectar la colisión entre Pacman y cualquier fantasma (tanto en la misma posición como en el cruce frontal directo en el mismo turno).
   b. Descontar una vida al ocurrir una colisión y reiniciar las posiciones iniciales de las entidades.

5. Ciclo de Partida y Reinicio:
   a. Declarar victoria al consumir la totalidad de los puntos presentes en el mapa.
   b. Declarar Game Over si el jugador pierde la totalidad de sus vidas.
   c. Permite reiniciar la partida de manera limpia para un nuevo jugador en cualquier momento.

6. Ranking de Puntuaciones:
   a. Listar en pantalla los mejores puntajes registrados (Top 5) leídos desde el archivo persistente.

REQUERIMIENTOS TÉCNICOS

1. Estructura del programa

Separación clara de responsabilidades en módulos independientes (modelo.py, persistencia.py, main.py).
Desacoplamiento total entre la lógica de negocio del juego y la interfaz gráfica de usuario.

2. Clases obligatorias a implementar:

Entidad (Clase Base Abstracta):
- Atributos: pos_x, pos_y, color
- Métodos: mover(), actualizar() (método abstracto)

Pacman (Subclase de Entidad):
- Atributos: direccion, vidas
- Métodos: perder_vida(), reiniciar_posicion(), actualizar()

Fantasma (Subclase de Entidad):
- Atributos: nombre, pos_inicial
- Métodos: reiniciar_posicion(), actualizar()

TableroJuego:
- Atributos: laberinto, puntaje, juego_terminado, victoria
- Métodos: actualizar_ciclo(), _reiniciar_posiciones_entidades()

GestorPuntuaciones:
- Atributos: nombre_archivo
- Métodos: guardar_puntuacion(), obtener_mejores_puntuaciones()

Se deberá aplicar:
Herencia entre Entidad y sus subclases (Pacman, Fantasma).
Polimorfismo redefiniendo el método actualizar() en las subclases.
Encapsulamiento: atributos privados/protegidos y uso de decoradores @property.
Abstracción mediante la clase base Entidad.

3. Diagrama UML

El diagrama debe reflejar el comportamiento del sistema representado por sus clases (atributos, métodos, visibilidad) y sus relaciones (herencia, composición, agregación).

4. Persistencia

Persistencia: Guardado automático de historial de puntuaciones en un archivo plano (TXT, CSV o JSON).

METODOLOGÍA Y CRONOGRAMA

El trabajo se desarrollará de manera progresiva y grupal, en equipos de hasta 3 integrantes. La implementación deberá realizarse por etapas, integrando gradualmente las funcionalidades y los conceptos de programación abordados durante la asignatura.

Se dispondrán clases de consulta y seguimiento para revisar los avances, resolver dificultades y orientar las decisiones de diseño e implementación.

Cada equipo deberá utilizar GitHub como herramienta de seguimiento y gestión del proyecto, manteniendo un repositorio actualizado que refleje la evolución del trabajo mediante commits frecuentes y descriptivos. El repositorio deberá incluir el código fuente y los demás entregables solicitados, y estará disponible para su revisión durante el desarrollo.

Se espera que todos los integrantes participen activamente en la implementación, comprendan las decisiones adoptadas y puedan justificar individualmente el funcionamiento del sistema durante la presentación y defensa oral.

Fechas Importantes
Consigna de Práctico: **21/11/2026**
Entrega del Práctico: **04/11/2026**
Presentación Oral: **11/11/2026**

ENTREGA FORMAL

Enlace directo al repositorio del proyecto. Dicho repositorio debe contener:

- Código fuente completo (.py).
- Diagrama UML en formato de imagen (.png, .jpg)
- Archivo README.md que incluya:
    - Nombres de los integrantes
    - Descripción breve del sistema
    - Instrucciones de ejecución

PRESENTACIÓN ORAL

Duración: 10 minutos (La presentación será bajo cronograma de horarios definidos por sorteo).
Deben explicar:

- Objetivo y alcance del proyecto.
- Estructura del código (módulos, clases, funciones).
- Decisiones de diseño (herencia, atributos, polimorfismo, encapsulamiento).

- Demostración del programa (Ejecutar el programa y probar cada funcionalidad en vivo). Explicación del Diagrama UML.
- Dificultades encontradas y aprendizajes.
- Evaluación (individual y grupal): Claridad expositiva, Dominio conceptual y Trabajo en equipo.